Algorítmica

Tema 5. Algoritmos para la exploración de grafos

Francisco Javier Cabrerizo Lorite

Curso académico 2024-2025

ETS de Ingenierías Informática y de Telecomunicación. Universidad de Granada

Índice

1. Introducción

2. Notación

3. Motivación: la suma de subconjuntos

4. La técnica de vuelta atrás (*Backtracking*)

5. Vuelta atrás: ejemplos de aplicación

6. La técnica de ramificación y poda (*Branch & Bound*)

7. Ramificación y poda: ejemplos de aplicación

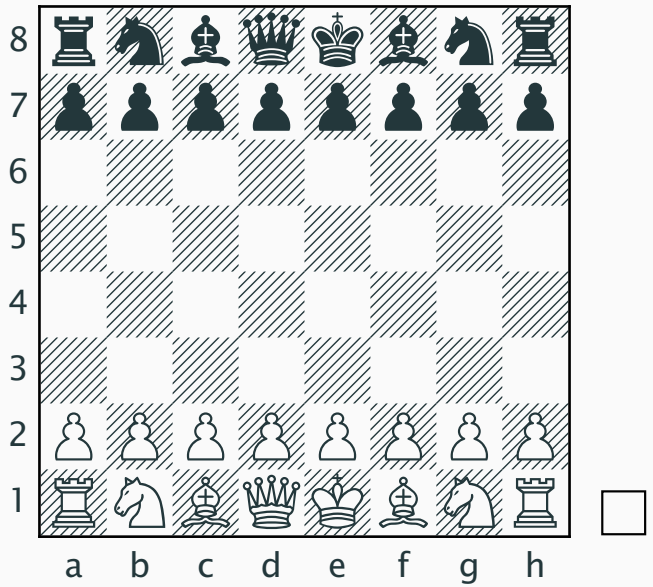
Objetivos

- Saber identificar soluciones distintas para un problema y evaluar su bondad.
- Ser consciente de la importancia del análisis de la eficiencia de un algoritmo como paso previo a su implementación.
- Entender al árbol de estados como una representación lógica del conjunto de todas las posibles soluciones de un problema.
- Conocer las técnicas de exploración de grafos (vuelta atrás y ramificación y poda) y su aplicación en la resolución de problemas, comprendiendo sus características principales y las diferencias entre ellas.
- Entender y saber aplicar el uso de cotas para reducir el espacio de búsqueda en las técnicas de exploración de grafos.

Introducción

Características de los problemas (1/2)

- Supongamos que debemos tomar un conjunto de decisiones entre varias alternativas donde:
  - No tenemos suficiente información sobre qué decisión tomar.
  - Cada decisión abre un abanico nuevo de alternativas.
  - Alguna secuencia de decisiones (incluso más de una) puede ser una solución del problema.



Características de los problemas (2/2)

- La solución se debe poder expresar mediante una tupla:

$$(x_1, x_2, \ldots, x_n)$$

  donde cada x_i se selecciona de un conjunto finito S_i.

- La solución consiste en encontrar:
  - La tupla que optimiza (maximiza o minimiza) una función objetivo o de criterio $P(x_1, x_2, \ldots, x_n)$.
  - Una tupla que satisfaga (no optimice) la función de criterio.
  - Todas las tuplas que satisfagan la función criterio.

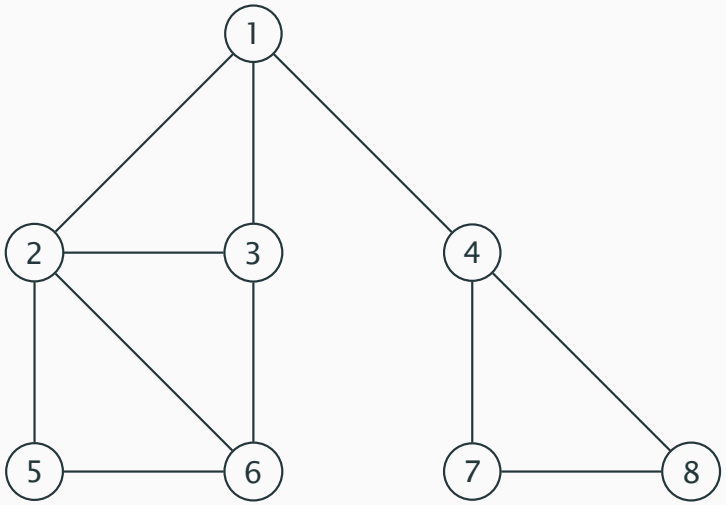
- Como muchos de estos problemas se pueden representar mediante un grafo, es necesario conocer las técnicas para su exploración:
  - Recorrido en profundidad.
  - Recorrido en anchura.

Recorrido en profundidad

```
procedimiento recorrido_profundidad(G = (V, A))
    para todo v ∈ V hacer
        marca[v] ← no visitado
    fin para todo
    para todo v ∈ V hacer
        si marca[v] = no visitado entonces
            rp(v)
        fin si
    fin para todo
fin procedimiento
```

```
procedimiento rp(vértice v)
    // v no ha sido visitado aún
    marca[v] ← visitado
    para todo w adyacente a v hacer
        si marca[w] = no visitado entonces
            rp(w)
        fin si
    fin para todo
fin procedimiento
```

Recorrido en profundidad



rp(1). Se visita el vértice 1 y pasa al adyacente.

rp(2). Llamada recursiva, visita 3.

rp(3). Llamada recursiva, visita 6.

rp(6). Llamada recursiva, visita 5.

rp(5). Sin vértices adyacentes.

rp(4). Llamada recursiva de `rp(1)`, visita 7.

rp(7). Llamada recursiva, visita 8.

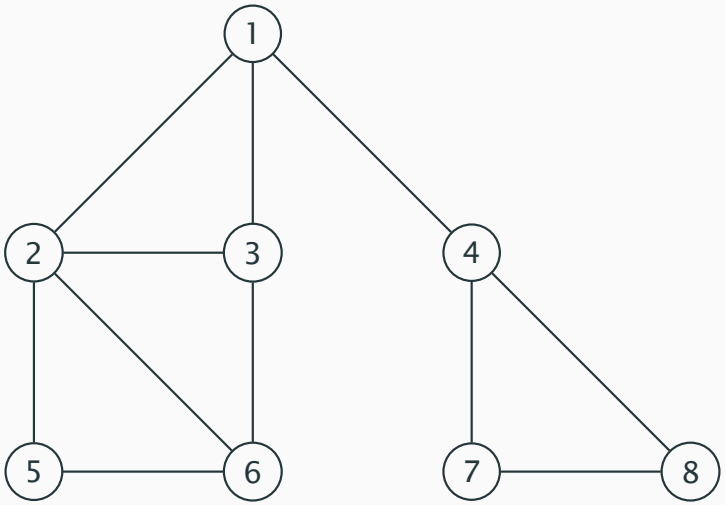
rp(8). Sin vértices que visitar.

Recorrido en anchura

```
procedimiento recorrido_anchura(G = (V, A))
    para todo v ∈ V hacer
        marca[v] ← no visitado
    fin para todo
    para todo v ∈ V hacer
        si marca[v] = no visitado entonces
            ra(v)
        fin si
    fin para todo
fin procedimiento
```

```
procedimiento ra(vértice v)
    Q ← cola vacía
    marca[v] ← visitado
    Insertar v en Q
    mientras Q ≠ ∅ hacer
        u ← primero(Q)
        Quitar u de Q
        para todo w adyacente a u hacer
            si marca[w] = no visitado entonces
                marca[w] ← visitado
            fin si
            Insertar w en Q
        fin para todo
    fin mientras
fin procedimiento
```

Recorrido en anchura



Paso	Visitado	Q
1	1	2, 3, 4
2	2	3, 4, 5, 6
3	3	4, 5, 6
4	4	5, 6, 7, 8
5	5	6, 7, 8
6	6	7, 8
7	7	8
8	8	

Fuerza bruta (búsqueda exhaustiva)

- Explora todas las posibles combinaciones de elementos.
- Sea m_i el tamaño del conjunto S_i, hay $m = m_1 m_2 \cdots m_n$ n-tuplas posibles candidatos a satisfacer la función objetivo o de criterio P.
- El enfoque basado en fuerza bruta trataría de formar todas esas tuplas y evaluar cada una de ellas en P, eligiendo la que diera un mejor valor de P o la que satisfaga P.

Alternativas a la fuerza bruta

- Las técnicas de vuelta atrás (*Backtracking*) y ramificación y poda (*Branch & Bound*) exploran los posibles estados de un problema hasta llegar a la solución.

- Pueden implementar un recorrido en profundidad, en anchura o variantes que ayuden en la búsqueda.

- La idea es construir la tupla eligiendo una componente cada vez y, mediante funciones de criterio modificadas $P_i(x_1, x_2, \ldots, x_n)$, comprobar si la tupla que se está creando puede llegar a ser una solución.

- La ventaja es que, si a partir de la tupla parcial $(x_1, x_2, \ldots, x_i)$ se deduce que no se podrá construir una solución, entonces se pueden ignorar por completo $m_{i+1} \cdots m_n$ posibles tuplas.

Notación

Notación

- **Solución parcial.** Tupla solución para la que aún no se han asignado valores a todas sus componentes.

- **Función de poda.** Aquella que permite identificar cuándo una solución parcial conduce (o no) a una solución del problema.

- **Restricciones explícitas.** Reglas que restringen a cada componente x_i de la tupla solución a tomar valores solo en un conjunto dado. Todas las tuplas que satisfacen las restricciones explícitas definen un espacio de soluciones de la instancia que se está resolviendo.

- **Restricciones implícitas.** Aquellas que determinan cuál (o cuáles) de las tuplas del espacio de soluciones satisface la función de criterio. Describen la forma en la que las x_i deben relacionarse entre sí. Son las restricciones que impone el camino que define la solución.

Notación

- **Árbol de estados.** Organización en árbol del espacio de soluciones.
- **Estado del problema.** Cada uno de los vértices del árbol de estados.
- **Estados solución.** Aquellos estados s del problema para los que el camino desde la raíz a s define una n-tupla en el espacio de soluciones.
- **Estados respuesta.** Aquellos estados solución s para los que el camino desde la raíz hasta s define una tupla que es miembro del conjunto de soluciones (satisfacen las restricciones implícitas) del problema.

Notación

- **Vértice vivo.** Vértice (estado del problema) que ha sido generado, pero del que aún no se han generado todos sus vértices hijos.

- **Vértice muerto (explorado).** Aquel que ha sido generado y, o bien se ha podado, o bien se han generado todos sus hijos.

- **Vértice en expansión (e-vértice).** Vértice vivo del que actualmente se están generando sus hijos.

Motivación: la suma de subconjuntos

La suma de subconjuntos

Enunciado

Dados $n + 1$ números enteros positivos, w_i, $1 \leq i \leq n$, y uno más M, se trata de encontrar todos los subconjuntos de números w_i cuya suma sea igual a M.

Ejemplo

- Sea $n = 4$, con $(w_1, w_2, w_3, w_4) = (11, 13, 24, 7)$, y $M = 31$.
- Los subconjuntos buscados son:

$$(11, 13, 7) \qquad\qquad (24, 7)$$

Pueden existir distintas representaciones de forma que todas las soluciones sean tuplas satisfaciendo ciertas restricciones

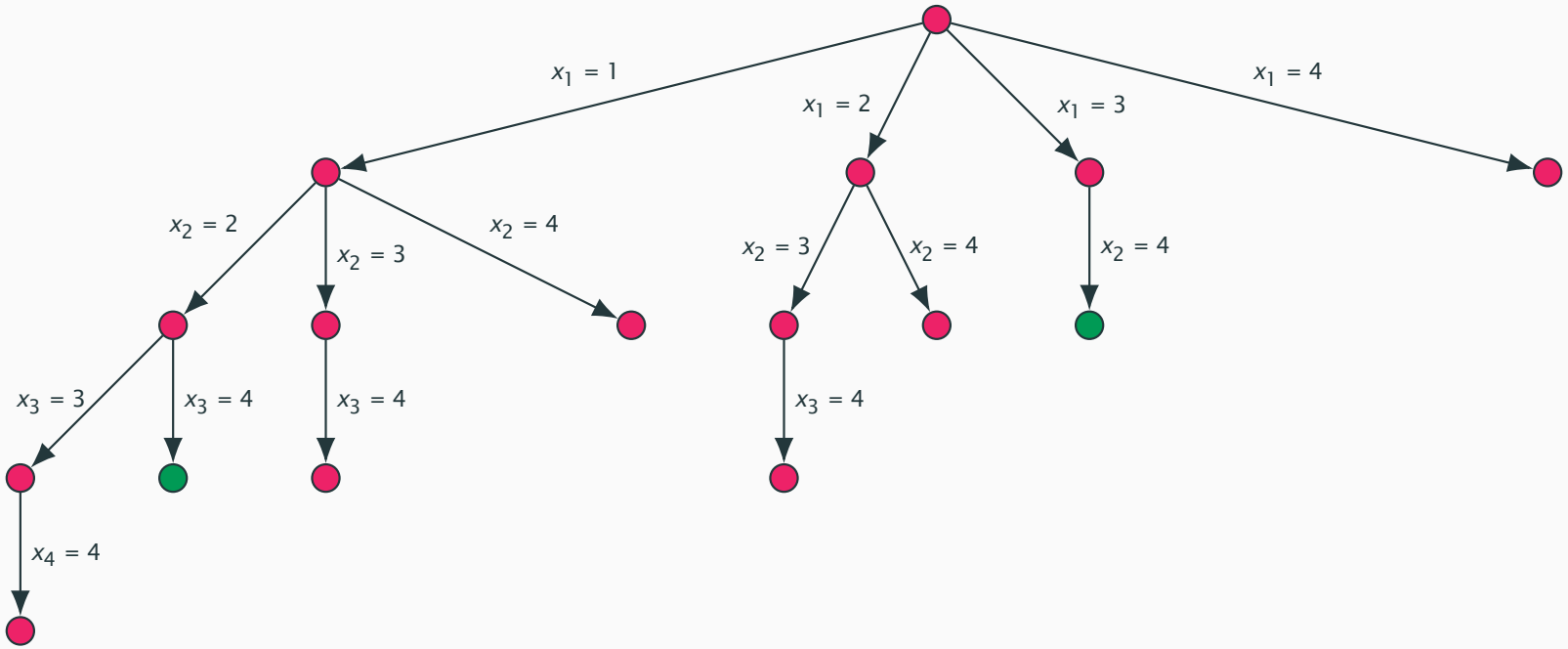
Representación: primera alternativa

- Una solución se puede representar mediante una tupla que contenga los índices correspondientes w_i.
- Todas las soluciones son k-tuplas $(x_1, x_2, \ldots, x_k)$, $1 \leq k \leq n$, y soluciones diferentes pueden tener tamaños de tupla diferentes.
- Restricciones explícitas:
  - $x_i \in \{j : \ j \text{ es entero y } 1 \leq j \leq n\}$.
- Restricciones implícitas:
  - No puede haber dos componentes con igual valor: $x_i \neq x_j$, para $i \neq j$.
  - La suma de los correspondientes w_i debe ser igual a M.
  - $x_i < x_{i+1}$, para $1 \leq i < n$. Por ejemplo, $(1, 2, 4)$ y $(1, 4, 2)$ representa el mismo subconjunto.

Árbol de estados: primera alternativa

- Los arcos se etiquetan de modo que uno que va desde el nivel de vértices i hasta el nivel de vértices $i + 1$ representa un valor para x_i. En esta representación, todos los vértices son estados solución.

- Los posibles caminos son (), (1), (1, 2), (1, 2, 3), (1, 2, 3, 4), (1, 2, 4), (1, 3), (1, 3, 4), (1, 4), (2), (2, 3), (2, 3, 4), (2, 4), (3), (3, 4), (4).



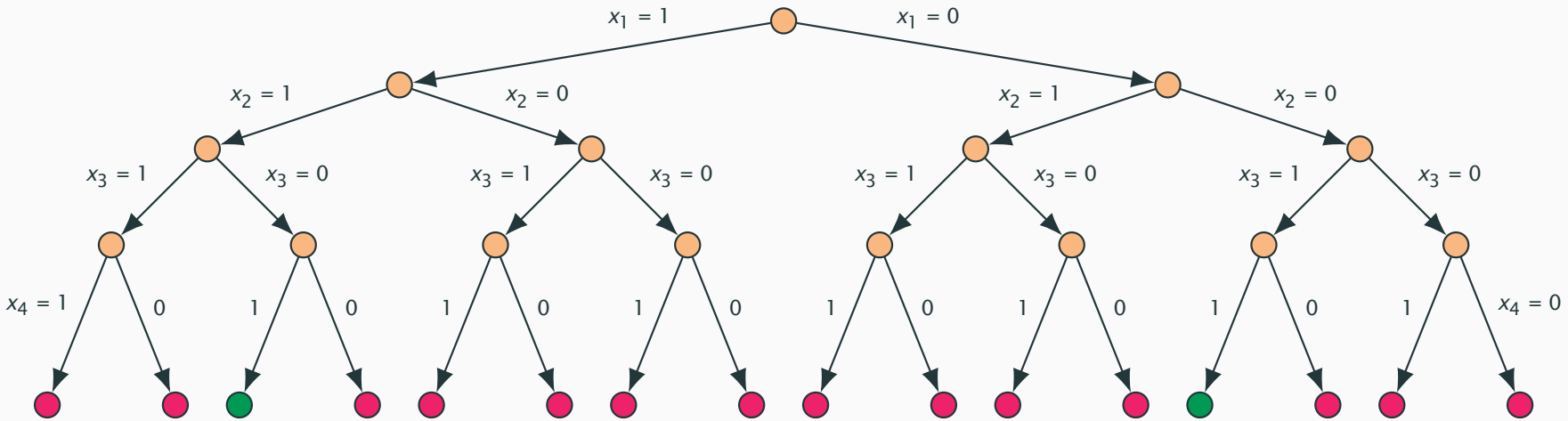
Representación: segunda alternativa

- Cada solución se representa por una n-tupla $(x_1, \ldots, x_n)$ tal que $x_i \in \{0, 1\}$, $1 \leq i \leq n$, de forma que:
  - Si $x_i = 1$, entonces se elige w_i.
  - Si $x_i = 0$, entonces no se elige w_i.
- Restricciones explícitas:
  - $x_i \in \{0, 1\}$, $1 \leq i \leq n$.
- Restricciones implícitas:
  - $\sum_{i=1}^{n} x_i w_i = M$.
- Esta formulación expresa todas las soluciones usando un tamaño de tupla fijo. Las soluciones del anterior caso son $(1, 1, 0, 1)$ y $(0, 0, 1, 1)$. Para ambas representaciones, el espacio solución consiste en 2^4 tuplas distintas.

Árbol de estados: segunda alternativa

- Un arco del nivel de vértices i al nivel de vértices $i + 1$ se etiqueta con el valor de x_i (0 o 1). En esta representación, solo los vértices hoja son estados solución.

- Todos los caminos desde la raíz a las hojas definen el espacio solución.



Resolución por fuerza bruta

- Implícitamente, se impone una estructura de árbol sobre el conjunto de posibles soluciones (espacio de soluciones).
- La generación de soluciones equivale a efectuar un recorrido del árbol, cuyos vértices (hojas) son posibles soluciones del problema.
- ¿Se puede mejorar el proceso? Sí, si se elimina la necesidad de llegar a todos los vértices del árbol.
- ¿Cuándo? Cuando desde un vértice no se puede alcanzar una solución. En tal caso, se puede podar la rama completa del árbol.
- ¿Cómo?
  - Vuelta atrás (*Backtracking*).
  - Ramificación y poda (*Branch & Bound*).

La técnica de vuelta atrás (*Backtracking*)

Características (1/2)

- Genera la solución de forma incremental. En cada paso, se añade un nuevo elemento.
- A partir de un vértice raíz, construye los diferentes estados del problema, manteniendo una lista de vértices vivos.
- El proceso de construcción es recursivo y equivale a un recorrido en profundidad del árbol de estados.
- Tan pronto como un nuevo hijo h del e-vértice en curso c ha sido generado, este se convierte en un nuevo e-vértice.
- El vértice c se convertirá de nuevo en e-vértice cuando el sub-árbol h haya sido explorado por completo.
- Emplean funciones de acotación para «matar» los vértices vivos sin tener que generar todos sus vértices hijos.

Características (2/2)

- Si un vértice solución s es respuesta, hay tres opciones:
  - El algoritmo termina, retornando s.
  - El algoritmo añade s al conjunto de soluciones y continúa el proceso de búsqueda.
  - El algoritmo evalúa la función objetivo en tal vértice, $P(s)$, actualiza la solución óptima s_{opt} y continúa el proceso de búsqueda.
- Si un vértice solución no es respuesta, el algoritmo continúa el proceso de búsqueda.
- Ello implica deshacer decisiones previas (volver a una llamada recursiva anterior) y tomar en su lugar otras decisiones que conducen a otras ramas del árbol y finalmente a nuevos vértices solución, que serán evaluados.

Implementación (1/2)

- Se supone que se quieren encontrar todos los vértices respuesta (y no solo uno).
- Sea $(x_1, x_2, \ldots, x_i)$ un camino desde la raíz hasta un vértice del árbol de estados.
- Sea $T(x_1, x_2, \ldots, x_i)$ el conjunto de todos los posibles valores x_{i+1} tales que $(x_1, x_2, \ldots, x_{i+1})$ también es un camino hacia un estado del problema.

Implementación (2/2)

- Existen funciones de acotación B_{i+1} (expresadas como predicados) tales que $B_{i+1}(x_1, x_2, \ldots, x_{i+1})$ es falsa para un camino $(x_1, x_2, \ldots, x_{i+1})$ desde el vértice raíz hasta un estado del problema si el camino no se puede extender para llegar a un vértice respuesta.

- $B_{i+1}(x_1, x_2, \ldots, x_{i+1})$ indica si $(x_1, x_2, \ldots, x_{i+1})$ satisface las restricciones implícitas del problema.

- Los candidatos para la posición $i + 1$ de la tupla solución $(x_1, x_2, \ldots, x_n)$ son aquellos valores generados por $T(x_1, x_2, \ldots, x_i)$ que satisfacen B_{i+1}.

Pseudocódigo

```
procedimiento backtracking(X[1..n], k)
    para todo X[k] ∈ T(X[1..k−1]) hacer
        si B_k(X[1..k]) = true entonces
            si X[1..k] es un camino hacia un vértice respuesta entonces
                imprimir(X[1..k])
            fin si
            backtracking(X[1..n], k + 1)
        fin si
    fin para todo
fin procedimiento
```

Eficiencia (1/2)

- Depende de cuatro factores:
  - El tiempo necesario para generar el siguiente x_k.
  - El número de valores x_k que satisfacen las restricciones explícitas.
  - El tiempo requerido por las funciones de acotación B_k.
  - El número de x_k que satisfacen las funciones B_k para todo k.
- Las funciones de acotación se consideran buenas si reducen considerablemente el número de vértices que hay que explorar.
- Las buenas funciones de acotación consumen mucho tiempo en su evaluación, por lo que hay que buscar un equilibrio entre el tiempo de evaluación y la reducción del número de vértices generados.

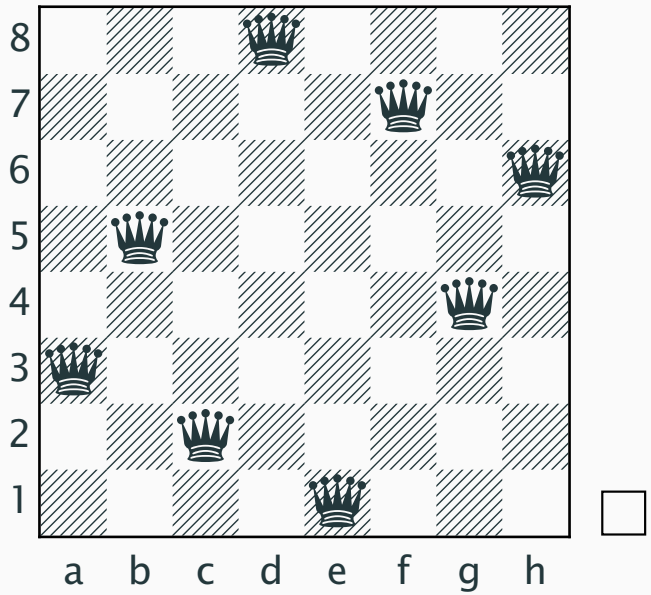
Eficiencia (2/2)

- De los cuatro factores, solo el cuarto varía de una instancia del problema a otra (el número de vértices generados).
- Para una instancia, un algoritmo *backtraking* podría generar solo $\mathrm{O}(n)$ vértices, mientras que en otra (parecida) podría generar casi todos los vértices del árbol de estados.
- Si el número de vértices en el espacio solución es 2^n o $n!$, el tiempo del peor caso para el algoritmo será generalmente $\mathrm{O}(p(n)2^n)$ u $\mathrm{O}(q(n)n!)$ respectivamente, con p y q polinomios en n.
- La importancia del *backtraking* reside en su capacidad para resolver instancias con grandes valores de n en muy poco tiempo.
- La dificultad está en predecir la conducta del algoritmo *backtraking* en la instancia concreta que se desea resolver.

Vuelta atrás: ejemplos de aplicación

Enunciado

Se considera un tablero de ajedrez de $n \times n$ casillas. El objetivo es colocar n reinas en el tablero sin que se amenacen entre sí. Una reina amenaza a las casillas de su misma fila, de su misma columna y de sus mismas diagonales.



Las n reinas

Representaciones

- Una n-tupla de las posiciones de las reinas, siendo cada posición un par (fila, columna).
  - Si $n = 8$, habría que examinar $\binom{64}{8} = 4\,426\,165\,368$ situaciones.
- Como cada reina debe estar en una fila diferente, podemos suponer que la reina i se coloca en la fila i. Las soluciones se representan como una n-tupla donde cada x_i es la columna en la que se coloca la reina i.
  - Si $n = 8$, habría que examinar $8^8 = 16\,777\,216$ situaciones.

Vuelta atrás: componentes de diseño (1/2)

- **Representación.** La solución se representa con una tupla $(x_1, x_2, \ldots, x_n)$ donde cada componente i representa una fila del tablero y cada valor x_i es la columna donde se colocará la reina de la i-ésima fila.

- **Restricciones explícitas:**
  - $x_i \in \{1, 2, \ldots, n\}, 1 \leq i \leq n$.

- **Restricciones implícitas:**
  - No puede haber dos reinas en la misma columna: $x_i \neq x_j$, $i \neq j$. Si $n = 8$, esto reduce el tamaño a $8! = 40\,320$.
  - No puede haber dos reinas en la misma diagonal: $|x_j - x_i| \neq |j - i|$, $i \neq j$.

Vuelta atrás: componentes de diseño (2/2)

- **Árbol de estados.** El espacio de estados es un árbol de grado n y altura $n + 1$ en el que cada vértice del nivel i tiene n hijos correspondientes a las n posibles asignaciones para x_i y en el que los vértices del nivel $n + 1$ son vértices hoja (estados solución o respuesta).

- **Función de criterio.** Función $P(x_1, x_2, \ldots, x_n)$ que se satisface para todas las tuplas $(x_1, x_2, \ldots, x_n)$ que cumplen que ninguna reina amenaza a otra, es decir, $x_i \neq x_j$, $i \neq j$, y $|x_j - x_i| \neq |j - i|$, $i \neq j$. Indica cuando una hoja del árbol es un estado respuesta.

- **Función de acotación.** Función $B_{i+i}(x_1, x_2, \ldots, x_{i+1})$ que poda el árbol si al añadir a la solución parcial, $(x_1, x_2, \ldots, x_i)$, el valor correspondiente de x_{i+1} no se satisfacen las restricciones implícitas: $x_i \neq x_j$, $i \neq j$, y $|x_j - x_i| \neq |j - i|$, $i \neq j$.

Vuelta atrás: pseudocódigo (1/2)

```
procedimiento reinas(X[1..n], k)
    si k = n entonces
        imprimir_solución(X[1..n])
    si no
        para todo columna = 1, . . . , n hacer
            X[k] ← columna
            si factible(X[1..n], k) entonces
                reinas(X[1..n], k + 1)
            fin si
        fin para todo
    fin si
fin procedimiento
```

Vuelta atrás: pseudocódigo (2/2)

```
función factible(X[1..n], k)
    válido ← true
    i ← 1
    mientras i < k && válido = true hacer
        si X[i] = X[k] || | k − i | = | X[k] − X[i] |  entonces
            válido ← false
        fin si
        i ← i + 1
    fin mientras
    devolver válido
fin función
```

El coloreo de un grafo

Enunciado

Sea $G = (V, A)$ un grafo no dirigido y m un número entero positivo. Queremos saber si los vértices del grafo se pueden colorear de forma que no haya dos vértices adyacentes con el mismo color y que solo se utilicen m colores para esa tarea.

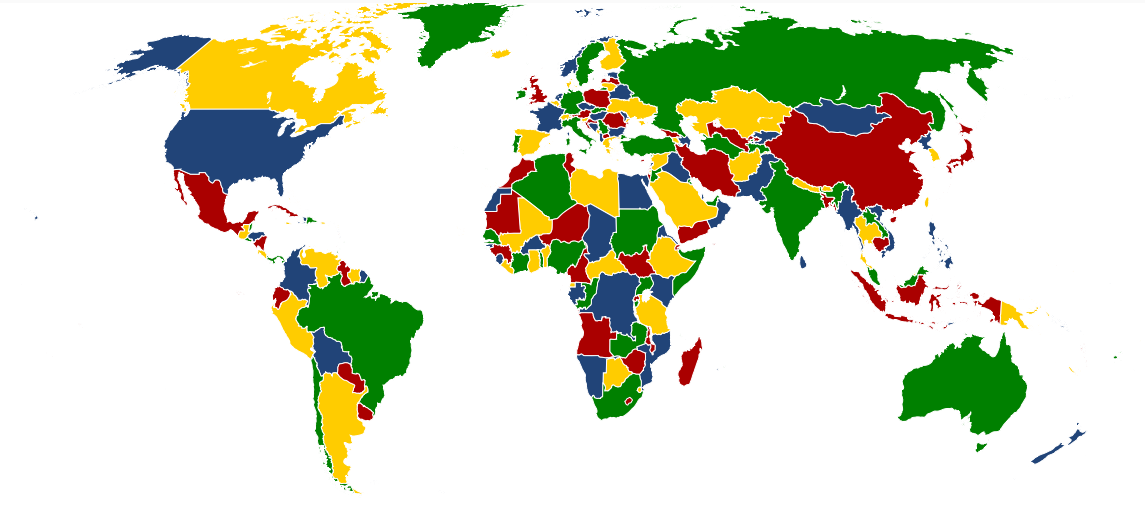
Se conoce como el problema de la m-colorabilidad.

El problema de optimización de la m-colorabilidad pregunta por el menor número m con el que el grafo se puede colorear. Al número entero m se le denomina número cromático del grafo.

El coloreo de un grafo

- Un grafo es plano si y solo si puede pintarse en un plano de modo que ningún par de aristas se corten entre sí.

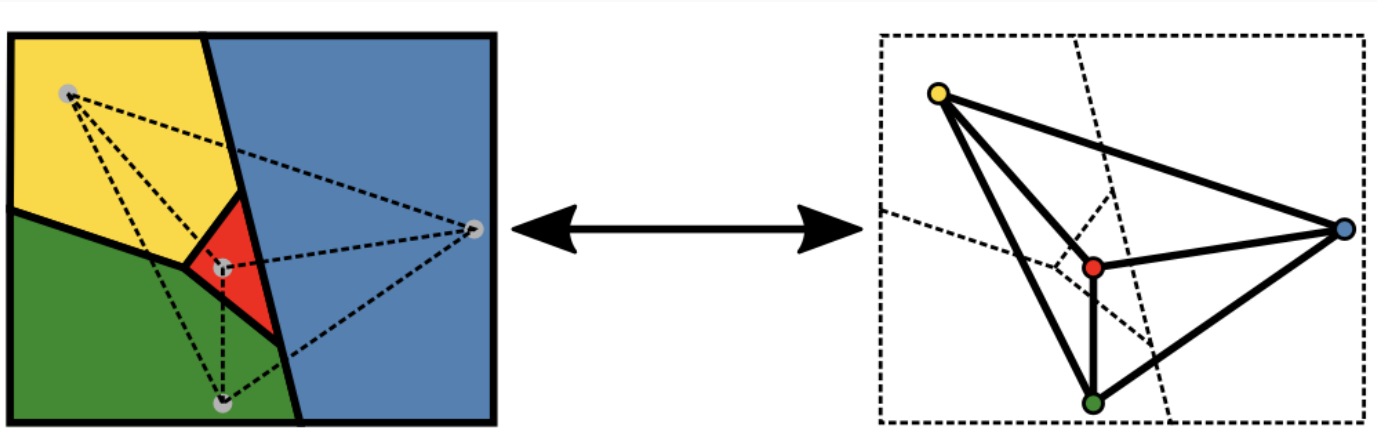
- Un caso especial y famoso del problema de la m-colorabilidad es el problema de los cuatro colores para grafos planos que, dado un mapa cualquiera, consiste en saber si ese mapa podrá pintarse con tan solo cuatro colores y de forma que no haya dos zonas colindantes con el mismo color (sí se puede).



El coloreo de un grafo

El coloreo de un mapa

- Queremos colorear un mapa con cuatro colores a lo sumo.
- Los países adyacentes deben tener colores diferentes.
- Este problema es fácilmente traducible a la nomenclatura de grafos.



Vuelta atrás: componentes de diseño (1/2)

- **Representación.** La solución se representa como una tupla $(x_1, x_2, \ldots, x_n)$ donde cada componente x_i contiene el color del vértice i, siendo n el número de vértices.
- **Restricciones explícitas:**
  - $x_i \in \{1, 2, \ldots, m\}$.
- **Restricciones implícitas:**
  - $x_i \neq x_j$ si i y j son vértices (países) adyacentes.

Vuelta atrás: componentes de diseño (2/2)

- **Árbol de estados.** El espacio de estados es un árbol de grado m y altura $n + 1$ en el que cada vértice del nivel i tiene m hijos correspondientes a las m posibles asignaciones para x_i y en el que los vértices del nivel $n + 1$ son vértices hoja (estados solución o respuesta).

- **Función de criterio.** Función $P(x_1, x_2, \ldots, x_n)$ que se satisface para todas las tuplas $(x_1, x_2, \ldots, x_n)$ que cumplen que no hay dos vértices adyacentes pintados con el mismo color, es decir, $x_i \neq x_j$ si i y j son vértices adyacentes. Indica cuando una hoja del árbol es un estado respuesta.

- **Función de acotación.** Función $B_{i+i}(x_1, x_2, \ldots, x_{i+1})$ que poda el árbol si al añadir a la solución parcial, $(x_1, x_2, \ldots, x_i)$, el valor correspondiente de x_{i+1} no se satisfacen las restricciones implícitas: $x_i \neq x_j$ si i y j son vértices adyacentes.

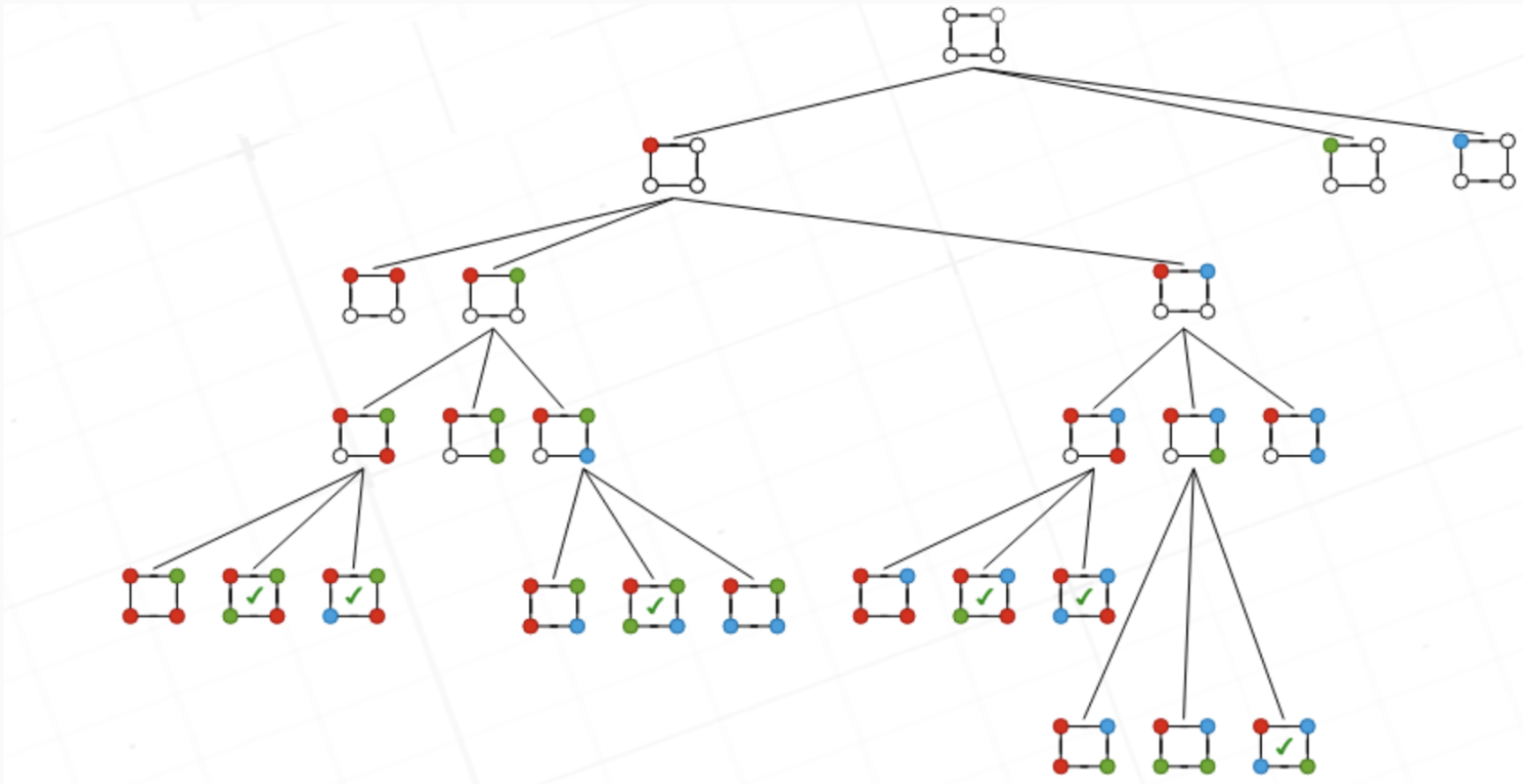
Vuelta atrás: pseudocódigo (1/2)

```
procedimiento coloreo(M[1..n][1..n], X[1..n], m, k)
    si k = n entonces
        imprimir_solución(X[1..n])
    si no
        para todo color = 1, . . . , m hacer
            X[k] ← color
            si factible(M[1..n][1..n], X[1..n], k) entonces
                coloreo(M[1..n][1..n], X[1..n], m, k + 1)
            fin si
        fin para todo
    fin si
fin procedimiento
```

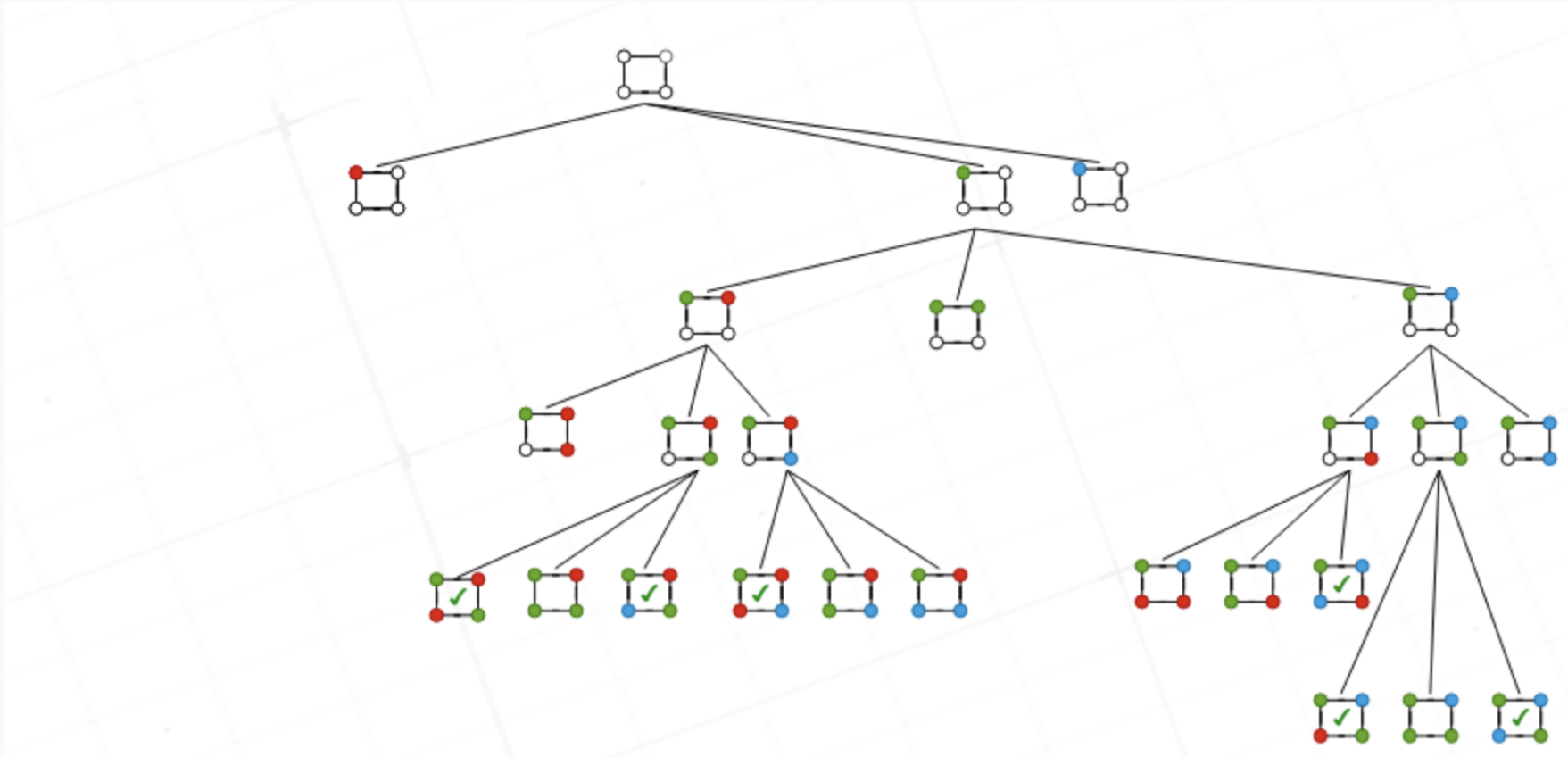
Vuelta atrás: pseudocódigo (2/2)

```
función factible(M[1..n][1..n], X[1..n], k)
    válido ← true
    i ← 1
    mientras i < k && válido = true hacer
        si M[k][i] = 1 && X[i] = X[k] entonces
            válido ← false
        fin si
        i ← i + 1
    fin mientras
    devolver válido
fin función
```

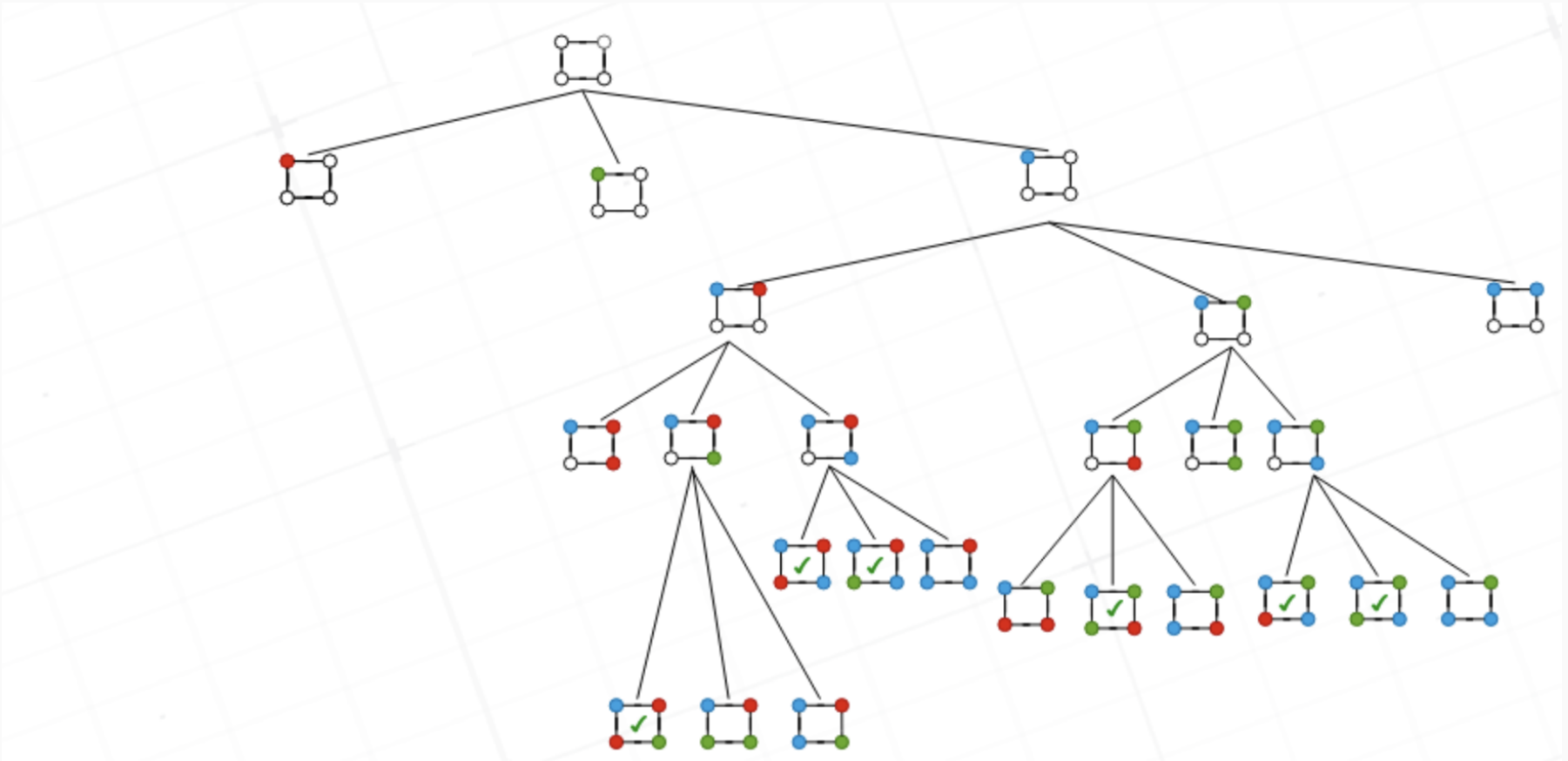
El coloreo de un grafo



El coloreo de un grafo

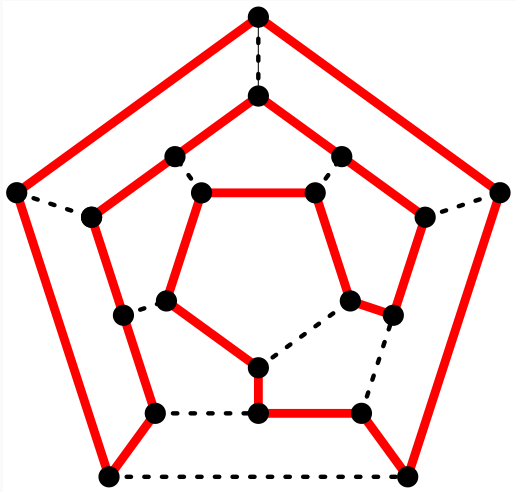


El coloreo de un grafo



Enunciado

Sea $G = (V, A)$ un grafo conexo de orden n. Se define un ciclo hamiltoniano como un camino cerrado simple que pasa por todos los vértices del grafo una sola vez. Se desean encontrar todos los ciclos hamiltonianos del grafo.



Vuelta atrás: componentes de diseño (1/2)

- **Representación.** La solución se representa mediante una tupla $(x_1, x_2, \ldots, x_n)$ donde cada componente x_i se asocia al vértice que hay que visitar justo en la i-ésima posición en el ciclo propuesto, siendo n el número de vértices.

- **Restricciones explícitas:**
  - $x_1 = 1$ (para evitar imprimir el mismo ciclo n veces).
  - $x_i \in \{2, 3, \ldots, n\}$, $i = 2, 3, \ldots, n$.

- **Restricciones implícitas:**
  - $x_i \neq x_j$, para $i \neq j$ (no puede visitarse el mismo vértice dos veces).
  - Debe existir una arista que una el vértice 1 con el vértice x_1.
  - Debe existir una arista que una los vértices x_i y x_{i+1}.
  - Debe existir una arista que una el vértice x_n (último vértice visitado del ciclo) con el vértice 1.

Vuelta atrás: componentes de diseño (2/2)

- **Árbol de estados.** El espacio de estados es un árbol de grado $n-1$ y altura n en el que cada vértice del nivel i tiene $n-1$ hijos correspondientes a las $n-1$ posibles asignaciones para x_i y en el que los vértices del nivel n son vértices hoja (estados solución o respuesta).

- **Función de criterio.** Función $P(x_1, x_2, \ldots, x_n)$ que se satisface para todas las tuplas $(1, x_2, \ldots, x_n)$ que cumplen que los vértices $1 - x_2 - \ldots - x_n - 1$ forman un ciclo hamiltoniano. Indica cuando una hoja del árbol es un estado respuesta.

- **Función de acotación.** Función $B_{i+i}(x_1, x_2, \ldots, x_{i+1})$ que poda el árbol si al añadir a la solución parcial, $(1, x_2, \ldots, x_i)$, el valor correspondiente de x_{i+1} no se satisfacen las restricciones implícitas.

Vuelta atrás: pseudocódigo (1/2)

```
procedimiento ciclos_hamiltonianos(M[1..n][1..n], X[1..n], k)
    si k = n entonces
        imprimir_solución(X[1..n])
    si no
        para todo vertice = 2, . . . , n hacer
            X[k] ← vertice
            si factible(M[1..n][1..n], X[1..n], k) entonces
                ciclos_hamiltonianos(M[1..n][1..n], X[1..n], k + 1)
            fin si
        fin para todo
    fin si
fin procedimiento
```

Vuelta atrás: pseudocódigo (2/2)

```
función factible(M[1..n][1..n], X[1..n], k)
    si k = n && M[x[n]][0] = ∞ entonces
        válido ← false
    si no si M[x[k−1]][x[k]] = ∞ entonces
        válido ← false
    si no
        válido ← true
        i ← 1
        mientras i < k && válido = true hacer
            si X[i] = X[k] entonces
                válido ← false
            fin si
            i ← i + 1
        fin mientras
    fin si
    devolver válido
fin función
```

La técnica de ramificación y poda (*Branch & Bound*)

Branch & Bound (B&B) (1/2)

- Es una técnica muy similar a la de *backtracking*.
- Basa su diseño en el análisis del árbol de estados de un problema:
  - Realiza un recorrido sistemático de ese árbol.
  - El recorrido no tiene que ser necesariamente en profundidad.
- Generalmente, se aplica para resolver problemas de optimización y para jugar a juegos.
- Los algoritmos diseñados con esta técnica tienen generalmente una complejidad de orden exponencial, o peor, en su peor caso.

Branch & Bound (B&B) (2/2)

- Su aplicación en casos muy grandes ha demostrado ser eficiente (incluso más que *backtracking*).
- Puede ser vista como una generalización (o mejora) de la técnica de *backtracking*.
- La principal novedad es que se considerará una estrategia de ramificación.
- Para eliminar vértices que no lleven a soluciones óptimas, un aspecto muy importante a considerar será el de las técnicas de poda.
- La poda se realiza estimando en cada vértice cotas del beneficio (coste) que podemos obtener a partir de este.

La técnica de ramificación y poda (*Branch & Bound*)

Diferencia con *backtracking*

- En *backtracking*, tan pronto como se genera un nuevo hijo del e-vértice actual, este hijo pasa a ser el nuevo e-vértice.

- En B&B, se generan todos los hijos del e-vértice antes de que cualquier otro vértice vivo pase a ser el nuevo e-vértice (no se realiza una búsqueda en profundidad).

En consecuencia

- En *backtracking*, los únicos vértices vivos son los que están en el camino de la raíz al vértice en curso.

- En B&B, puede haber más vértices vivos, los cuales se guardan en una estructura de datos auxiliar denominada lista de vértices vivos.

Más diferencias con *backtracking*

- En *backtracking*, el test de comprobación realizado por las funciones de acotación nos indica únicamente si un vértice concreto nos puede llevar a una solución o no.

- En B&B, se acota el valor de la solución a la que nos puede conducir un vértice concreto. Esta acotación permite:
  - Podar el árbol, si sabemos que no nos va a conducir a una solución mejor que la que tenemos actualmente.
  - Establecer el orden de ramificación, de modo que comenzaremos explorando las ramas más prometedoras del árbol.

Descripción general (1/2)

- Explora un árbol de estados comenzando a partir de un problema raíz (el problema original con su espacio de soluciones completo).
- Entonces, aplica procedimientos de cota inferiores y superiores al problema raíz.
- Si las cotas cumplen las condiciones que se hayan establecido, habremos encontrado la solución óptima y el procedimiento termina.
- Si no, el espacio de soluciones se divide en dos o más, dando lugar a distintos subproblemas.
- Estos subproblemas particionan el espacio de soluciones. La búsqueda se desarrolla en cada uno de ellos.
- El método se aplica recursivamente a los subproblemas.

Descripción general (2/2)

- Si se encuentra un solución óptima para un subproblema, esta será una solución factible para el problema completo, pero no necesariamente el óptimo global.
- Cuando en un vértice (subproblema) la cota superior es menor que el mejor valor conocido en la región, no puede existir un óptimo global en el subespacio de la región factible asociada a ese vértice.
- Por tanto, ese vértice puede ser eliminado (podado) en posteriores consideraciones.
- La búsqueda sigue hasta que se examinan o podan todos los vértices, o hasta que se alcanza algún criterio preestablecido sobre el mejor valor encontrado y las cotas superiores de los subproblemas no resueltos aún.

La técnica de ramificación y poda (*Branch & Bound*)

Para cada vértice *i* se tiene:

- Una **cota superior**, $CS(i)$, y una **cota inferior**, $CI(i)$, del beneficio (o coste) óptimo que se puede alcanzar a partir de ese vértice.
  - Determinan cuándo se puede realizar una poda.
- Una **estimación del beneficio (o coste)** óptimo que se puede encontrar a partir de ese vértice. Puede ser una media de las anteriores.
  - Ayuda a decidir qué parte del árbol evaluar primero.
- Una **estrategia de poda:**
  - Se supone un problema de maximización.
  - Se han recorrido varios vértices $1, 2, \ldots, n$, estimando para cada uno la cota superior $CS(j)$ y la cota inferior $CI(j)$, respectivamente, para $j \in \{1, \ldots, n\}$.
  - Hay dos casos.

Caso 1

- Si a partir de un vértice i se puede obtener una solución válida, entonces podar tal vértice si:

$$CS(i) \leq CI(j) \qquad \text{para algún vértice } j \text{ generado en el árbol.}$$

- **Ejemplo.** Problema de la mochila usando un árbol binario:
  - A partir de i, podemos encontrar un beneficio máximo de $CS(i) = 4$.
  - A partir de j, se garantiza un beneficio mínimo de $CI(j) = 5$.
  - Podemos podar i sin perder ninguna solución óptima.

Caso 2

- Si se obtiene una posible solución válida para el problema con un beneficio $B(j)$, entonces podemos podar un vértice i si:

$$CS(i) \leq B(j) \qquad \text{para algún vértice } j\text{, solución final (factible).}$$

- **Ejemplo.** Problema de la mochila usando un árbol binario:
  - A partir de i, podemos encontrar un beneficio máximo de $CS(i) = 4$.
  - Se ha encontrado una solución válida cuyo beneficio es 6.
  - Podemos podar i sin perder ninguna solución óptima.

Estrategias de ramificación (1/2)

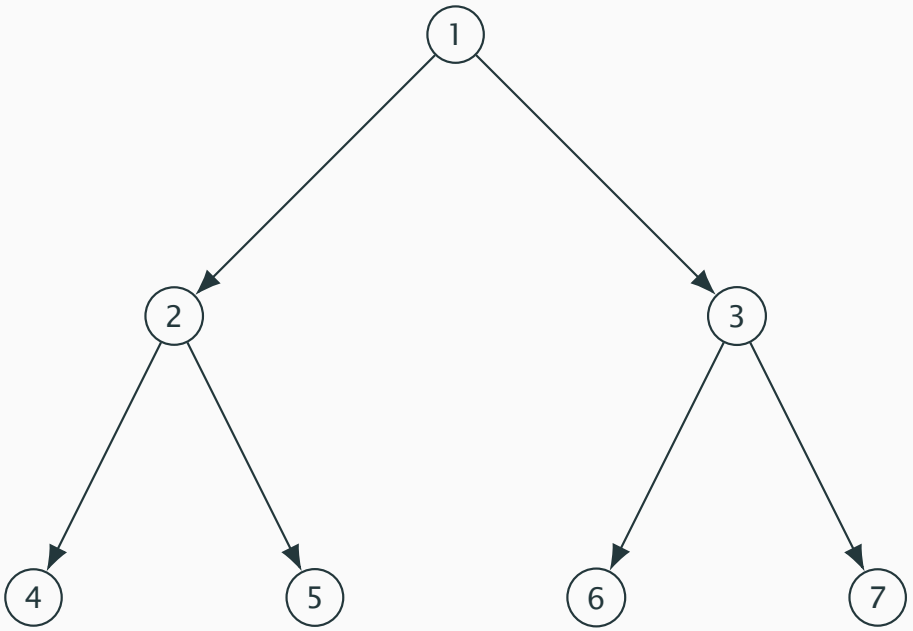
- Normalmente, el árbol de estados es implícito, no se almacena en ningún lugar.
- Para realizar el recorrido, se usa una lista de vértices vivos (LVV).
- La lista de vértices vivos contiene todos los vértices que han sido generados pero que no han sido explorados todavía (vértices pendientes de tratar por el algoritmo).
- Según como se implemente la lista de vértices vivos, el recorrido será de un tipo u otro.

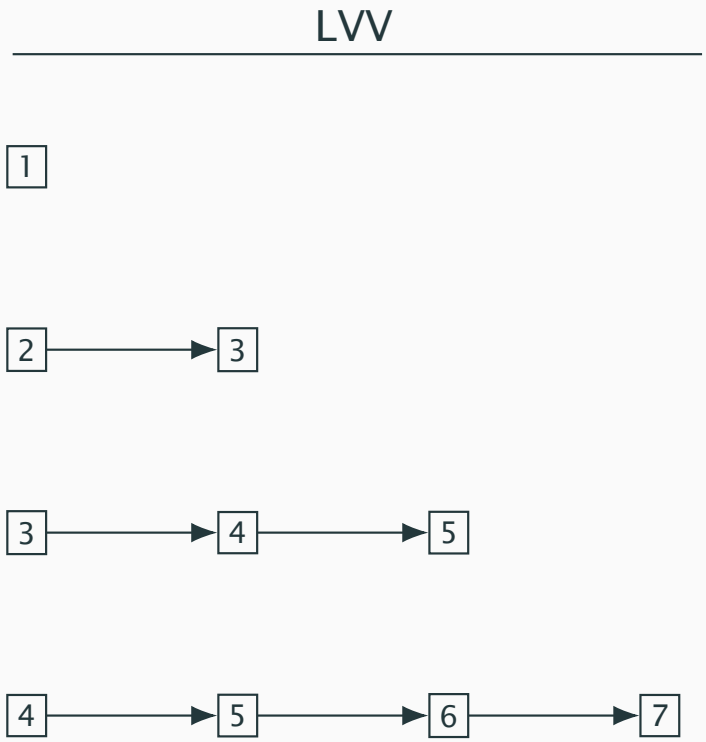
Estrategias de ramificación (2/2)

- Estrategias que no tienen en cuenta los beneficios (costes), realizan una búsqueda «a ciegas»:
  - **Estrategia FIFO** (First In, First Out).
  - **Estrategia LIFO** (Last In, First Out).
- Estrategias que usan estimaciones de beneficios (costes), exploran primero los vértices más prometedores:
  - **Estrategias LC** (Least Cost).
  - **Estrategias MB** (Maximum Benefit).

Estrategia de ramificación FIFO

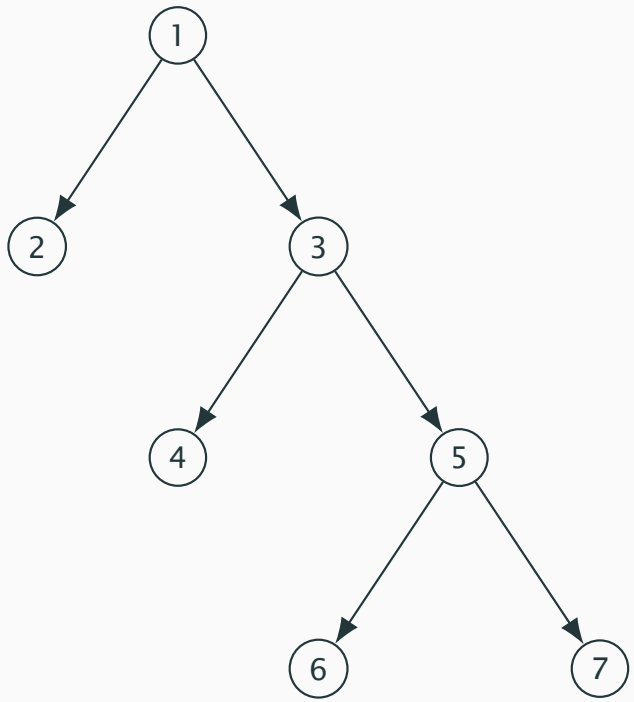
- La lista de vértices vivos es una cola.
- El recorrido del árbol se realiza en anchura.

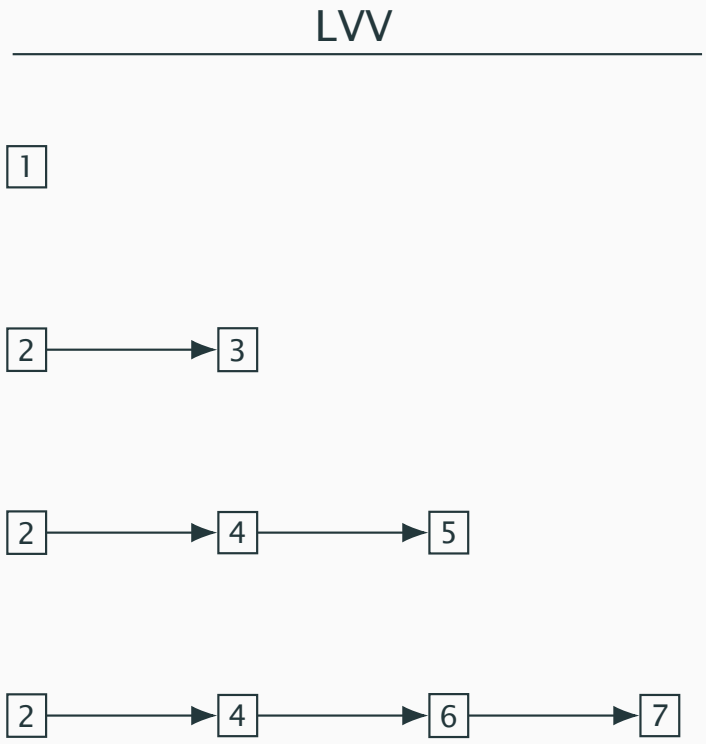




Estrategia de ramificación LIFO

- La lista de vértices vivos es una pila.
- El recorrido del árbol se realiza en profundidad.



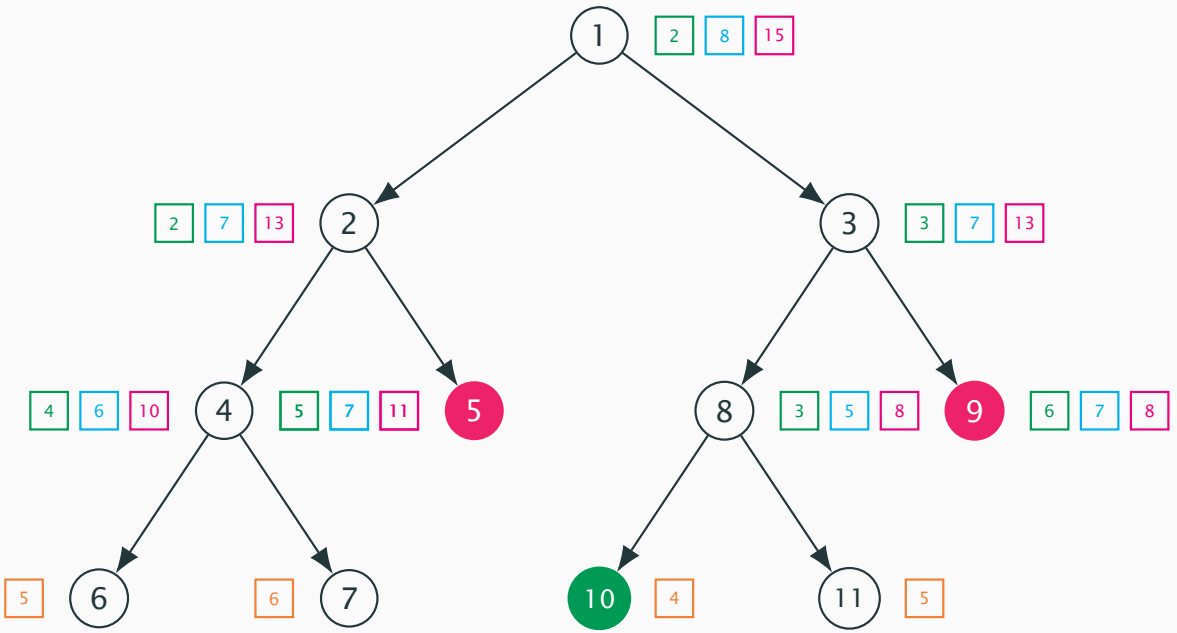


Estrategias de ramificación LC / MB

- En caso de empate (de beneficio o coste estimado), este se deshace usando un criterio FIFO o LIFO:
  - **Estrategia LC-FIFO / MB-FIFO.** Seleccionar el primero que se introdujo en la LVV (de los que empatan).
  - **Estrategia LC-LIFO / MB-LIFO.** Seleccionar el último que se introdujo en la LVV (de los que empatan).
- En cada vértice se puede tener una cota inferior del coste (beneficio), un coste (beneficio) estimado, y cota superior del coste (beneficio).
- Podar según los valores de las cotas (CS, CI).
- Ramificar según los valores estimados (E).

Ejemplo: B&B usando LC-FIFO (minimización)

- Para realizar la poda se usa una variable C, que contiene el valor de la menor de las cotas superiores hasta ese momento o de alguna solución final ya encontrada. Si para algún vértice i, $CI(i) \geq C$, entonces se poda el vértice i.



C	LVV
15	1
13	2 → 3
10	4 → 3 → 5
5	3 → 5
5	8 → 5
4	5

Pseudocódigo (minimización)

```
procedimiento branch_and_bound(vértice_raiz, solución)
    LVV ← {vértice_raiz}
    C ← CS(vértice_raiz)
    solución ← ∅
    mientras LVV ≠ ∅ hacer
        x ← seleccionar(LVV) // criterio FIFO, LIFO, LC-FIFO, LC-LIFO
        LVV ← LVV −{x}
        si CI(x) < C entonces // si no se cumple, se poda x
            para todo y hijo de x hacer
                si y es una solución final mejor que solución entonces
                    solución ← y
                    C ← mínimo(C, coste(y))
                si no si y no es solución final y CI(y) < C entonces
                    LVV ← LVV + {y}
                    C ← mínimo(C, CS(y))
                fin si
            fin para todo
        fin si
    fin mientras
fin procedimiento
```

Observaciones

- Solo se comprueba el criterio de poda cuando se introduce o se saca un vértice de la lista de vértices vivos.
- Si un hijo de un vértice es una solución final, entonces no se introduce en la lista de vértices vivos.
- Se comprueba si esa solución es mejor que la actual y, en tal caso, se actualiza C y se guarda como mejor solución hasta el momento.

Tiempo de ejecución

- Depende de:
  - **El número de vértices recorridos.** Depende de la efectividad de la poda.
  - **El tiempo empleado en cada vértice.** Tiempo necesario para hacer las estimaciones de coste y gestionar la lista de vértices vivos en función de la estrategia de ramificación.
- En el peor caso, el tiempo de ejecución de un algoritmo B&B es igual al de un algoritmo de *backtracking* (o peor si tenemos en cuenta el tiempo que requiere el manejo de la lista de vértices vivos).
- En el caso promedio, no obstante, se suelen obtener mejoras con respecto a *backtracking*.

¿Cómo hacer que un algoritmo B&B sea más eficiente?

- Hacer estimaciones de coste muy precisas:
  - Se realiza una poda exhaustiva del árbol de estados.
  - Se recorren menos vértices.
  - Se emplea mucho tiempo en realizar las estimaciones.
- Hacer estimaciones de coste poco precisas:
  - Se emplea poco tiempo en cada vértice.
  - No se poda mucho.
  - El número de vértices explorados puede ser muy elevado.

Conclusión

- Se debe buscar un equilibrio entre la precisión de las cotas y el tiempo empleado en calcularlas.

Ramificación y poda: ejemplos de aplicación

Enunciado

Se tienen n trabajadores y n tareas. Sea b_{ij} el beneficio de asignar la tarea j al trabajador i. Una asignación de tareas puede expresarse como una asignación de los valores 0 y 1 a las variables x_{ij}, donde:

- $x_{ij} = 0$ significa que al trabajador i no le han asignado la tarea j.
- $x_{ij} = 1$ indica que sí se la han asignado.

Una asignación es válida si a cada trabajador solo le corresponde una tarea y cada tarea está asignada a un único trabajador.

Dada una asignación válida, se define su beneficio como:

$$\sum_{i=1}^{n} \sum_{j=1}^{n} x_{ij} b_{ij}$$

Una asignación es óptima si es de máximo beneficio.

Ramificación y poda: componentes de diseño (1/3)

- **Representación.** La solución se representa mediante una tupla $(x_1, x_2, \ldots, x_n)$ donde el i-ésimo elemento, x_i, indica la tarea asignada al trabajador i, siendo n el número de trabajadores y de tareas.

- **Restricciones explícitas:**
  - $x_i \in \{1, 2, \ldots, n\}$, $i = 1, 2, \ldots, n$.

- **Restricciones implícitas:**
  - $x_i \neq x_j$, para $i \neq j$ (no puede asignarse la misma tarea a dos trabajadores distintos).

Ramificación y poda: componentes de diseño (2/3)

- **Árbol de estados.** El espacio de estados es un árbol de altura n que, comenzando por el vértice raíz en donde todavía no se ha asignado ninguna tarea a ningún trabajador, generará $n - i$ hijos para cada vértice en el nivel i, que se corresponden con las posibles asignaciones para x_i. Los vértices del nivel n serán los vértices hoja (estados respuesta).

- **Función objetivo.** Representa el beneficio total de asignar cada tarea a un trabajador distinto:

$$\sum_{i=1}^{n} b_{ix_i}$$

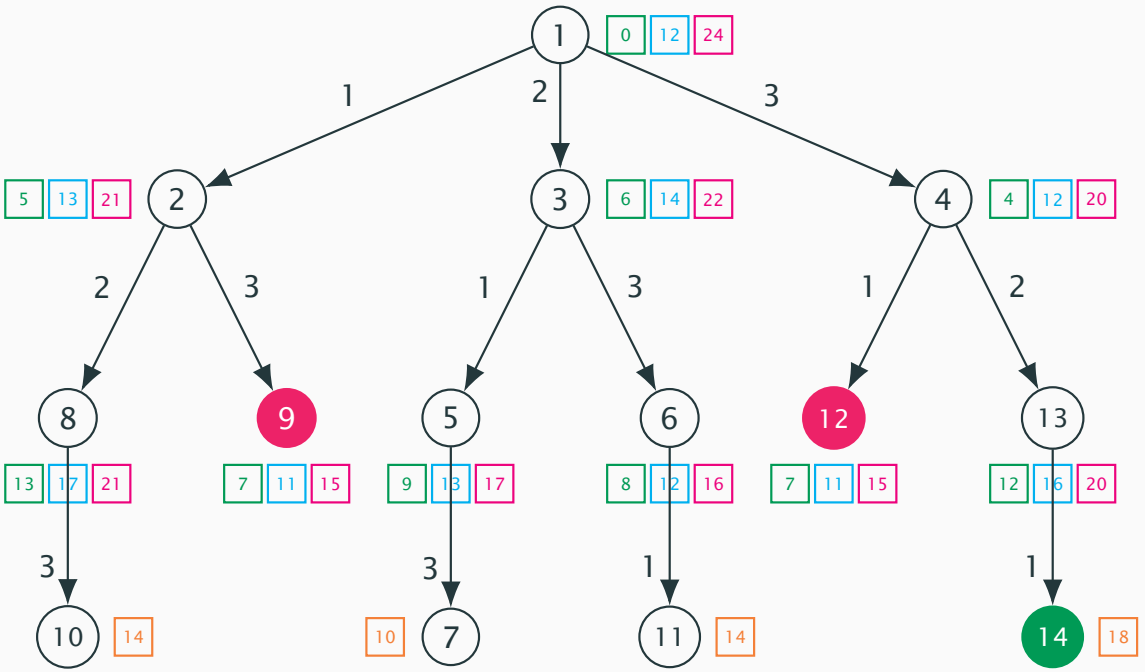
  El objetivo es obtener la tupla $(x_1, x_2, \ldots, x_n)$ que produce el máximo valor para esta función objetivo.

Ramificación y poda: componentes de diseño (3/3)

- **Cálculo de cotas:**
  - Cota inferior CI. Beneficio acumulado hasta ese momento.
  - Cota superior CS. Cota inferior más las restantes asignaciones con el máximo global.
  - Estimación del beneficio. Media de las cotas.
- **Estrategias de ramificación y poda (maximización):**
  - Variable de poda C. Valor de la mayor cota inferior o solución final del problema encontrada hasta ahora.
  - Condición de poda. Podar el vértice i si $CS(i) \leq C$.
  - Estrategia de ramificación. Estrategia MB-LIFO.

Ejemplo

B	1	2	3
1	5	6	4
2	3	8	2
3	6	5	1



C	LVV
0	1
6	3 → 2 → 5
9	5 → 2 → 6 → 4
10	2 → 6 → 4
13	8 → 6 → 4 → 9
14	6 → 4 → 9
14	4 → 9
14	13 → 9 → 12
18	9 → 12
18	12
18	

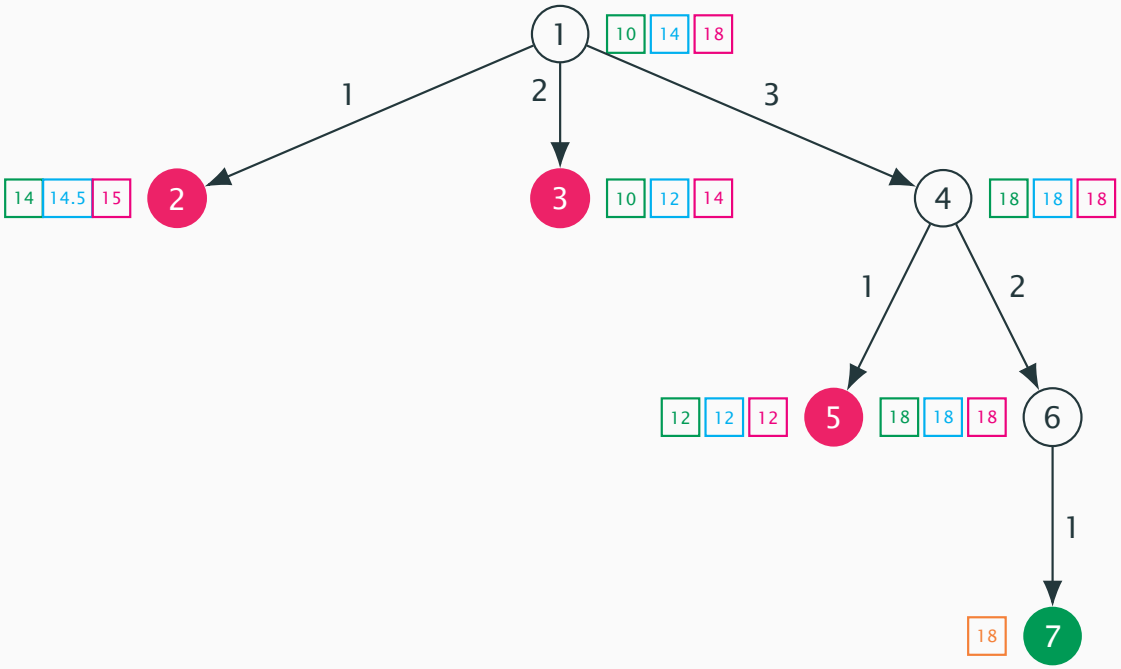
Cálculo de cotas mejoradas

- **Cota inferior *CI*.** Beneficio acumulado hasta ese momento más el resultado de asignar a cada persona la tarea libre que proporciona un mayor beneficio.
- **Cota superior *CS*.** Asignar las tareas con mayor beneficio (aunque estas se repitan).
- **Estimación del beneficio.** Media de las cotas.

La asignación de tareas

Ejemplo

B	1	2	3
1	5	6	4
2	3	8	2
3	6	5	1



C	LVV
10	1
18	4 → 2 → 3
18	6 → 2 → 5 → 3
18	2 → 5 → 3
18	5 → 3
18	3
18	

Solución: (3,2,1)

Beneficio: 18

El viajante de comercio

Enunciado

Se conocen las distancias entre un determinado número de ciudades. Partiendo de una de ellas, un viajante debe visitar cada ciudad exactamente una vez y regresar al punto de partida habiendo recorrido en total la menor distancia posible.

Enunciado formal

Dado un grafo $G = (V, A)$ no dirigido, conexo, ponderado y completo, y dado uno de sus vértices v_0, se desea encontrar el ciclo hamiltoniano de coste mínimo que comienza y termina en v_0.

Ramificación y poda: componentes de diseño (1/3)

- **Representación:** La solución se representa mediante una tupla $(x_1, x_2, \ldots, x_n)$ donde cada componente x_i se asocia con el vértice que hay que visitar en la i-ésima posición, siendo n el número de vértices del grafo (ciudades).

- **Restricciones explícitas:**
  - $x_1 = 1$ (se asume que se comienza por el vértice 1).
  - $x_i \in \{2, \ldots, n\}$, $i = 2, \ldots, n$.

- **Restricciones implícitas:**
  - $x_i \neq x_j$, para $i \neq j$ (no puede visitarse el mismo vértice más de una vez).

Ramificación y poda: componentes de diseño (2/3)

- **Árbol de estados.** El espacio de estados es un árbol de altura $n-1$ que, comenzando por el vértice raíz en el que se ha asignado $x_1 = 1$, generará $n-i$ hijos para cada vértice en el nivel i, que se corresponden con las posibles ciudades a las que se puede ir desde x_i (ya que no se pueden repetir). Los vértices del nivel $n-1$ serán los vértices hoja (estados respuesta), ya que en este punto se puede asignar directamente el valor a x_n (solo queda uno).

- **Función objetivo.** Coste total del ciclo hamiltoniano determinado por la solución $(x_1, x_2, \ldots, x_n)$:

$$\sum_{i=1}^{n-1} m_{x_i x_{i+1}} + m_{x_n x_1}$$

  donde m_{ij} da el coste de viajar de la ciudad (vértice) i a la ciudad (vertice) j. El objetivo es obtener la tupla $(x_1, x_2, \ldots, x_n)$ que produce el mínimo valor para esta función objetivo.

Ramificación y poda: componentes de diseño (3/3)

- **Cálculo de cotas:**
  - Cota inferior CI. El mejor coste será el de la arista adyacente que tenga el menor valor. La suma de los costes asociados a las mejores aristas incidentes en cada vértice aún por incluir en el ciclo hamiltoniano más el coste del camino ya recorrido ofrece una estimación válida para tomar decisiones.
- **Estrategias de ramificación y poda (maximización):**
  - Variable de poda C. Valor de la menor cota superior o solución final del problema encontrada hasta ahora.
  - Condición de poda. Podar el vértice i si $CS(i) \geq C$.
  - Estrategia de ramificación. Estrategia LC-LIFO.

Ejemplo

- Dada la siguiente matriz de adyacencias, ¿cuál es el coste mínimo del ciclo hamiltoniano que comienza y termina en el vértice 1?

$$M = \begin{bmatrix} 0 & 14 & 4 & 10 & 20 \\ 14 & 0 & 7 & 8 & 7 \\ 4 & 5 & 0 & 7 & 16 \\ 11 & 7 & 9 & 0 & 2 \\ 18 & 7 & 17 & 4 & 0 \end{bmatrix}$$

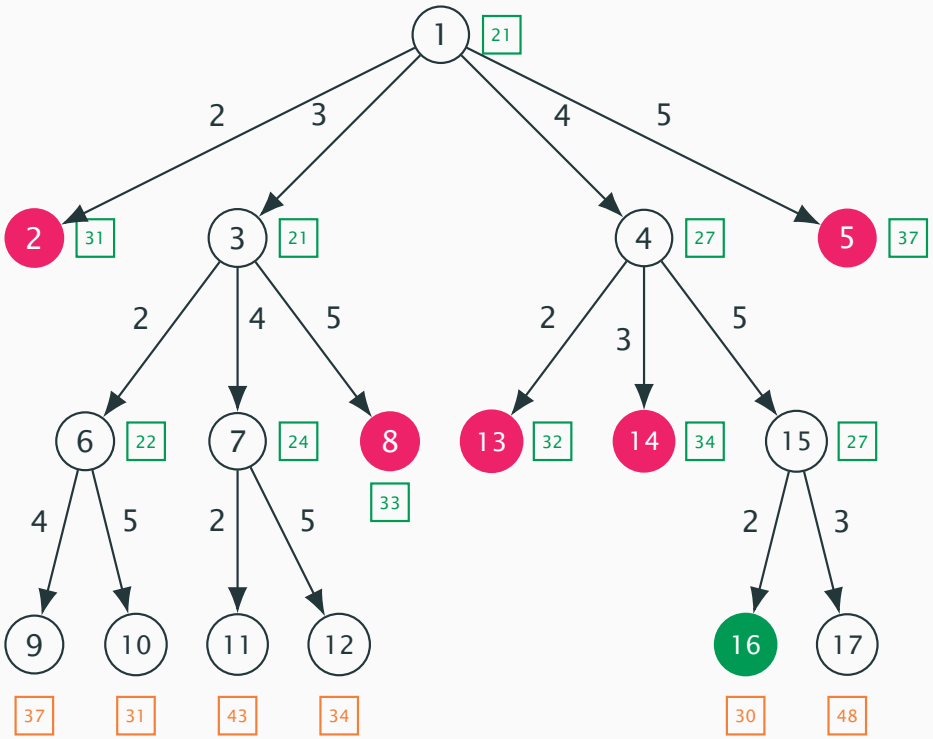
- Se aplica el algoritmo voraz para obtener una primera solución del problema, el cual obtiene $1 - 3 - 2 - 5 - 4 - 1$, con un coste de 31.

- Por tanto, $C = 31$.

El viajante de comercio

Ejemplo

$$\begin{bmatrix} 0 & 14 & 4 & 10 & 20 \\ 14 & 0 & 7 & 8 & 7 \\ 4 & 5 & 0 & 7 & 16 \\ 11 & 7 & 9 & 0 & 2 \\ 18 & 7 & 17 & 4 & 0 \end{bmatrix}$$



C	LVV
31	1
31	3 → 4
31	6 → 7 → 4
31	7 → 4
31	4
31	15
30	

Solución: 1-4-5-2-3-1

Coste: 30

Bibliografía

Bibliografía

Aclaración

- El contenido de las diapositivas es esquemático y representa un apoyo para las clases teóricas.

- Se recomienda completar los contenidos del tema 5 con apuntes propios tomados en clase y con la bibliografía principal de la asignatura.

Por ejemplo


G. Brassard and P. Bratley.
***Fundamentals of Algorithmics*.**
Prentice Hall, Englewwod Cliffs, New Jersey, 1996.


T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein.
***Introduction to Algorithms*.**
The MIT Press, Cambridge, MA, 2022.


J. L. Verdegay.
***Lecciones de Algorítmica*.**
Editorial Técnica AVICAM, 2017.